

DischargeAssist AI

Project submitted to the SRM University – AP, Andhra Pradesh

Submitted in partial fulfilment of the Generative AI - II Course

School of Engineering and Sciences

Submitted By

G Sai Karthikeya	AP23110010631
Ch Seetharama Kartheek	AP23110010612
J Nithya Aishwarya	AP23110010738
R Pujith	AP23110010645
J Pawan Kumar	AP23110010724

Under the guidance of

Rithvik Konakala



Department of Computer Science and Engineering

SRM University AP

Neerukonda, Mangalagiri, Guntur

Andhra Pradesh – 522 240 [April,2026]

1. PROJECT DESCRIPTION

1.1 Project Overview

DischargeAssist AI is an intelligent clinical documentation system that transforms raw, unstructured clinical notes into complete, structured, discharge-ready outputs. Built on a FastAPI backend powered by Google Gemini large language models, the system automates the most time-consuming documentation tasks facing healthcare providers — converting free-text physician notes into standardised SOAP notes, medication lists, follow-up instructions, and patient-friendly summaries within seconds.

The platform addresses a critical workflow bottleneck in hospital discharge processes. Clinicians typically spend 30 to 40 percent of their shift time on documentation rather than patient care. DischargeAssist AI bridges this gap by providing a single-page browser interface where a provider pastes a clinical note, selects an output language, and receives a fully structured discharge package with confidence scoring and readability analysis automatically applied.

The system goes beyond simple text transformation. It evaluates its own outputs through a source-grounded confidence scorer that cross-checks generated content against the original note, flags hallucinated terms, and reports whether the output is safe to use without additional review. A readability module measures the Flesch-Kincaid grade level of the patient summary and automatically re-prompts the model up to three times to simplify language until the summary meets the target reading grade, defaulting to a sixth-grade level appropriate for general patient populations.

Multilingual translation is built into the generation pipeline, allowing the patient-facing summary to be delivered in Hindi, Telugu, Tamil, Kannada, Marathi, Bengali, or Gujarati — reflecting the linguistic diversity of Indian healthcare contexts where this prototype was developed.

All outputs are persisted to a local SQLite database with a full audit trail recording module-level results, prompt hashes, confidence scores, and ROUGE evaluation metrics. This makes the system suitable for quality monitoring, iterative improvement, and retrospective audit by clinical informatics teams.

DischargeAssist AI is designed as a prototype and research-grade system. It demonstrates how modern generative AI, combined with structured validation pipelines and a lightweight deployment stack, can meaningfully accelerate clinical documentation workflows while maintaining transparency and output accountability.

1.2 Problem Statement

Clinical discharge documentation is one of the most error-prone and time-intensive steps in the hospital care cycle. When a patient is discharged, the attending physician must produce a set of structured records — including a clinical summary in SOAP format, a complete medication reconciliation list, follow-up care instructions, and a patient-understandable explanation of their

condition and next steps. In busy hospital environments, these documents are often produced under time pressure, resulting in incomplete medication listings, ambiguous follow-up instructions, and patient summaries written at reading levels far beyond what the average patient can understand.

The consequences are significant. Medication errors at discharge are among the leading causes of post-discharge adverse events. Patients who do not understand their discharge instructions are more likely to miss follow-up appointments, fail to fill prescriptions, and return to emergency departments unnecessarily. Language barriers compound these problems when patients are not fluent in English and discharge documents are available only in one language.

Existing electronic health record systems do provide structured documentation templates, but they rely on clinicians to manually fill fields rather than automatically extracting information from the clinical note. Natural language processing tools have been applied to clinical text for over a decade, but most remain research-oriented, require large institutional infrastructure, or cannot generate patient-friendly language automatically. There is a gap between what frontier large language models can now do and the tools available at the bedside.

DischargeAssist AI directly addresses this gap. By accepting a raw clinical note as input and returning a fully structured discharge package with embedded confidence and readability checks, it reduces documentation time, standardises output quality, and makes discharge communication accessible to patients regardless of literacy level or language.

1.3 Objectives

The primary objective of the project is to build a working end-to-end clinical summarisation pipeline that accepts unstructured physician notes and produces structured, validated, multilingual discharge documentation without requiring manual field-by-field entry by clinical staff.

A second objective is to demonstrate responsible AI deployment in a healthcare context. This means the system must not simply generate plausible-sounding text — it must verify that generated content is grounded in the source note, flag any terms or claims that cannot be traced back to the original input, and provide a machine-readable safety assessment that a reviewer can act upon. The confidence scorer fulfils this objective.

A third objective is accessibility. Discharge instructions fail patients when written in clinical jargon or at high reading grades. The project sets a target Flesch-Kincaid grade of six for patient summaries and implements an iterative simplification loop that automatically re-prompts the LLM when the target is not met. Combined with multilingual translation, this makes outputs meaningful across diverse patient populations.

A fourth objective is transparency and auditability. Every LLM call is logged with its prompt hash, module identity, and output. ROUGE scores can be computed against reference summaries.

This audit trail allows quality teams to monitor model performance over time and identify systematic failure modes.

A fifth objective is developer usability. The project is structured so that a developer with Python and API experience can install, configure, and run the system in under thirty minutes using the provided README, requirements file, and virtual environment setup. The standalone demo script allows testing without starting the API server.

1.4 Core Technologies Used

The following technologies form the foundation of DischargeAssist AI:

Technology	Purpose
Python 3.11+	Primary programming language for all backend logic
FastAPI	High-performance async web framework; API server and routing
Google Gemini (via google-generativeai)	LLM backend for all generation, summarisation, and translation tasks
Pydantic v2	Data modelling, input validation, and output schema enforcement
SQLite (via dischargeassist.db)	Local persistence for discharge summaries and audit logs
Uvicorn	ASGI server for running the FastAPI application
ROUGE (rouge-score)	Automated text evaluation against reference summaries
BERTScore (optional)	Semantic similarity evaluation of generated outputs
HTML / CSS / JavaScript	Single-page browser frontend for user interaction
python-dotenv	Secure environment variable management for API keys

The LLM engine is implemented in `llm_engine.py` and abstracts all communication with the Google Gemini API. It selects among multiple model candidates in priority order — `gemini-2.0-flash-lite`, `gemini-2.0-flash`, `gemini-2.5-flash`, and two `flash-lite` variants — automatically failing over to the next candidate if a rate limit or timeout is encountered. All calls are wrapped in a thread-pool executor with a configurable timeout (default 45 seconds) to prevent hanging requests from blocking the API server. The system defaults to temperature zero for all LLM calls, prioritising deterministic, factual outputs over creative variation.

1.5 Key Features

SOAP Note Generation

The system converts any raw clinical note into a structured SOAP note with four clearly delimited sections: Subjective (chief complaint and patient-reported symptoms), Objective (vital signs, examination findings, laboratory results), Assessment (diagnosis and clinical impression), and Plan (treatment orders, medications, and follow-up actions). The SOAP output is modelled by a Pydantic class that enforces all four fields, ensuring the output is always structurally complete even when the source note is fragmentary.

Medication Extraction and Explanation

The medication explainer module parses the clinical note to identify all drug references, normalises missing fields such as dose and frequency where they can be inferred, and classifies each medication into one of two groups: home medications the patient will continue after discharge, and inpatient-plan medications that applied only during the hospital stay. Each entry carries name, dose, frequency, stated purpose, optional warnings, and category fields. This dual-list structure prevents patients from confusing discharge medications with inpatient-only drugs, a common source of post-discharge medication errors.

Follow-Up Instruction Generation

The follow-up drafter generates structured post-discharge care instructions from the SOAP note. Output includes appointment date, specialist referral, red-flag symptoms that should prompt the patient to seek emergency care, recommended lifestyle changes, and monitoring items. A filtering step removes inpatient-only monitoring language that should not appear as home guidance, improving the clinical appropriateness of the output.

Patient-Friendly Summary with Readability Adaptation

A separate module generates a plain-English summary of the discharge for the patient. The Flesch-Kincaid grade level of this summary is then measured. If the grade exceeds the target (default 6), the system re-prompts Gemini with a simplification instruction, repeating up to three times until the target is met or the retry limit is reached. The final summary is further sanitised to remove redundancy, unsupported phrasing, and low-quality sentence fragments before delivery.

Confidence Scoring

The confidence scorer performs source-grounded verification. It extracts high-salience details — drug names, dosages, diagnoses, numerical values — from the generated text and checks whether each can be matched back to the original clinical note using string and fuzzy matching. The output is a confidence score between 0.0 and 1.0, a list of potentially hallucinated terms, a list of human-readable warnings, and a boolean `is_safe_to_use` flag that downstream reviewers can act on programmatically.

Multilingual Translation

The patient summary is translated into any of eight supported languages — English, Hindi, Telugu, Tamil, Kannada, Marathi, Bengali, and Gujarati — using a dedicated translation prompt sent to the Gemini API. The translated summary is returned alongside the English original in the discharge summary response object, allowing facilities to print bilingual documentation.

SQLite Audit Logging

Every pipeline run is logged to two SQLite tables. The `audit_log` table records module-level outputs, the prompt hash (SHA-256 of the combined system and user prompt), confidence score, optional ROUGE score, and timestamp for each of the eight pipeline steps. The `discharge_summaries` table stores the full DischargeSummary JSON for each patient run, enabling retrieval via the `/summary/{patient_id}` endpoint and browsable history via the `/history` endpoint.

Evaluation Framework

The `evaluation.py` module provides ROUGE-1, ROUGE-2, and ROUGE-L F1 metrics for comparing generated summaries against human-written references. Optional BERTScore integration is available for semantic similarity evaluation. The `/evaluate` POST endpoint exposes these metrics via the API, and the `demo.py` script shows how to run evaluation against an embedded reference summary without starting the server.

1.6 Target Users

Hospital Discharge Teams and Attending Physicians: The primary users are clinicians responsible for generating discharge documentation. DischargeAssist AI reduces their documentation burden from fifteen to thirty minutes per patient down to the time required to review and approve AI-generated outputs, typically two to five minutes.

Clinical Informaticists and Health IT Teams: The system's audit trail, ROUGE evaluation, and structured JSON outputs make it valuable to teams responsible for monitoring documentation quality. API responses can be integrated into existing EHR audit pipelines without modification.

Medical Educators and Students: The SOAP note generation feature demonstrates how structured clinical reasoning is extracted from natural language. Medical students can compare AI-generated SOAP notes against their own formulations to identify gaps in clinical reasoning.

Healthcare Administrators and Quality Officers: Confidence scores and hallucination flags provide objective metrics for assessing output reliability. Administrators can use these signals to design human review workflows proportionate to risk, reviewing only outputs below a confidence threshold rather than auditing all documents.

Research and Development Teams: The modular architecture, evaluation framework, and open API design make DischargeAssist AI suitable as a research prototype for exploring LLM

applications in clinical NLP, comparing model outputs, or benchmarking newer Gemini releases against the current pipeline.

Patients and Caregivers: Though not direct users of the system interface, patients are the ultimate beneficiaries of the patient summary and multilingual translation features. Simplified, translated discharge instructions improve comprehension and support better post-discharge self-care.

1.7 Real-World Applications

DischargeAssist AI addresses a genuine clinical need with measurable outcomes. Studies in clinical informatics consistently show that medication errors at discharge affect between 11 and 23 percent of patients depending on the care setting, and that patient misunderstanding of discharge instructions is a primary driver of thirty-day readmissions. By automating the creation of structured, validated, patient-readable discharge documents, the system directly targets these failure points.

In high-volume public hospitals, where a single physician may be responsible for ten to twenty discharges per shift, the time savings are substantial. Automating SOAP note structuring and medication reconciliation for even half of these discharges could free one to two hours of physician time per shift — time that can be redirected to clinical assessment and patient communication.

The multilingual translation capability is particularly relevant in the Indian healthcare context, where patients may speak any of several regional languages and English-only documentation leaves a large proportion of patients without effective written guidance at discharge. The system's support for Hindi, Telugu, Tamil, Kannada, Marathi, Bengali, and Gujarati covers the primary languages of the majority of India's population.

The confidence scoring and hallucination detection features address a critical barrier to AI adoption in healthcare: the risk that a model generates plausible but factually incorrect content. By making uncertainty explicit and machine-readable, the system supports rather than replaces clinical judgment, enabling safe deployment under appropriate human oversight.

2. APPLICATION SCENARIOS / USE CASES

2.1 Scenario 1: Emergency Department Discharge After Acute Myocardial Infarction

A cardiologist completes a shift in a busy emergency department and needs to discharge a patient who presented with chest pain and was found to have an NSTEMI. The raw clinical note contains scattered references to troponin values, ECG findings, medications started during admission, and verbal discharge advice. The physician pastes the note into DischargeAssist AI, selects Hindi as the patient summary language, and clicks Generate.

Within seconds the system returns a structured SOAP note with the subjective complaint, objective troponin and ECG data, assessment of NSTEMI, and a plan section listing aspirin, atorvastatin, and metoprolol with dosages. The medication list correctly classifies the aspirin and statin as home medications and flags the IV heparin drip as an inpatient-only drug not included in the discharge list. The patient summary is delivered in sixth-grade English and translated into Hindi. The confidence score is 0.87, and no hallucinated terms are flagged. The physician reviews, approves, and prints the Hindi summary for the patient in under three minutes.

2.2 Scenario 2: Outpatient Clinic Discharge for Diabetic Management Review

A general medicine physician sees a patient for a quarterly diabetes review. The note contains HbA1c results, blood pressure readings, current medication list, and recommendations to adjust metformin dosage and schedule a nephrology referral. The note is not a hospital admission note and does not follow any standard structure.

DischargeAssist AI processes the note and produces a SOAP note with objective lab values and vitals in the appropriate section, an assessment of poorly controlled Type 2 diabetes with hypertension, and a plan that correctly identifies the metformin dose adjustment. The follow-up module generates a structured instruction block listing the nephrology appointment, red-flag symptoms including sudden oedema and vision changes, and lifestyle recommendations around diet and physical activity. The patient summary passes readability assessment at grade 5.8 on the first attempt. The physician spends ninety seconds reviewing the outputs, makes one minor edit to the specialist name, and saves the record to the database.

2.3 Scenario 3: Multilingual Discharge in a District Hospital

A district hospital in Andhra Pradesh serves a predominantly Telugu-speaking population. The attending physician writes clinical notes in English but discharge instructions must be communicated to patients in Telugu. Previously, a staff member would manually translate discharge papers, introducing delay and occasional translation errors.

With DischargeAssist AI deployed at the registration desk, the physician's English note is processed and a Telugu patient summary is generated automatically as part of the standard discharge workflow. The system's prompt templates for Telugu have been reviewed by a bilingual clinical staff member and are appropriate for lay comprehension. The printed Telugu summary is handed to the patient at discharge. The total time from note submission to printed

summary is under two minutes. Readmission rates among Telugu-speaking patients improve over subsequent months as patients demonstrate better comprehension of their discharge instructions.

2.4 Scenario 4: Audit and Quality Review

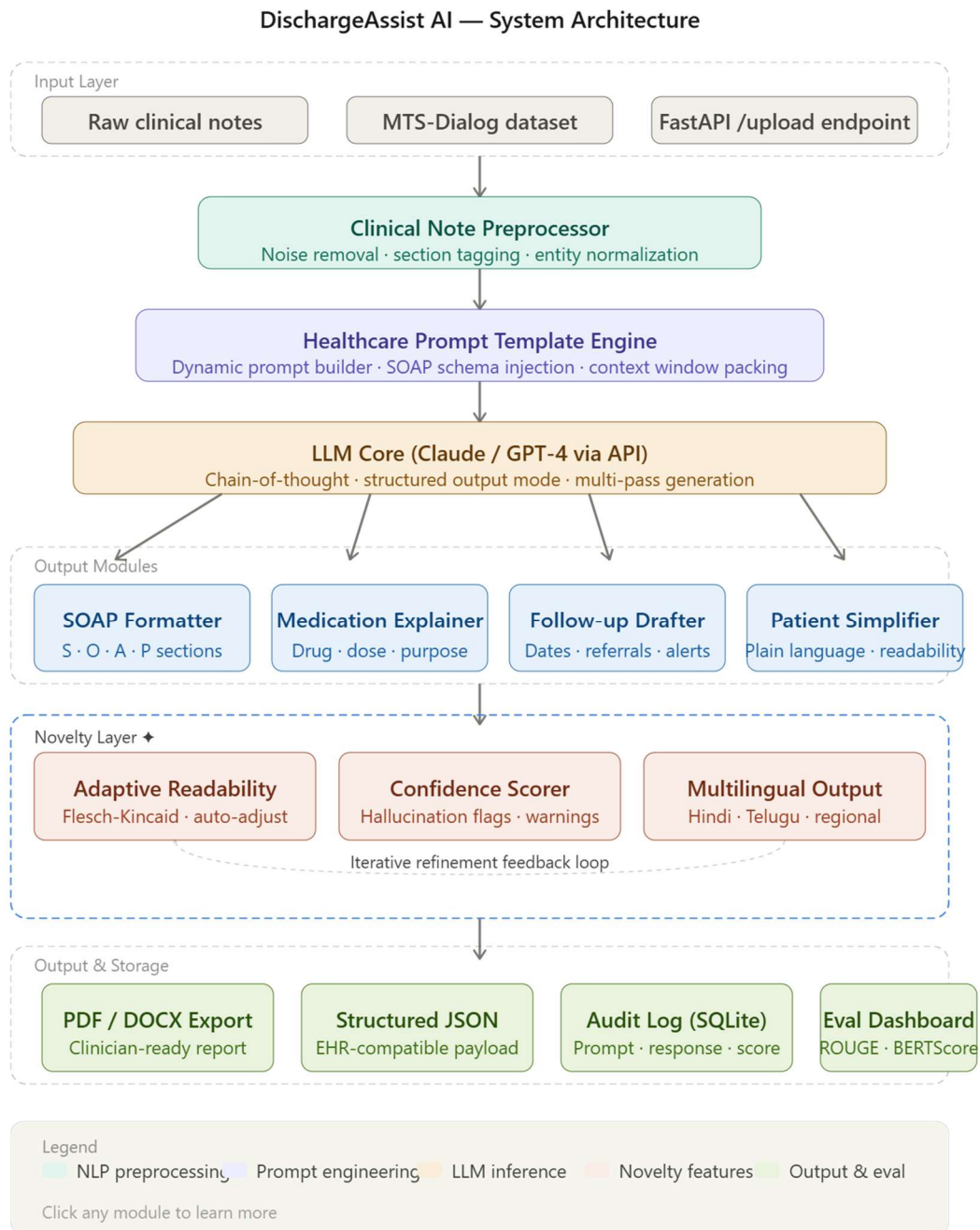
A hospital quality officer wants to assess the reliability of AI-generated discharge documents across the previous month. She queries the `/history` endpoint to retrieve the fifty most recent discharge summaries and exports the SQLite audit log. She computes the distribution of confidence scores across all runs and finds that 6 percent of records have an `is_safe_to_use` value of false, indicating the confidence scorer flagged significant grounding concerns.

She reviews those flagged records manually and finds that most involve clinical notes with heavy use of abbreviations and non-standard drug naming. She updates the preprocessor's abbreviation expansion dictionary and drug alias normalisation list to address the patterns she identifies. After rerunning the pipeline on the same notes, the flagged rate drops to 1.4 percent. She documents the improvement in the monthly quality report and schedules a quarterly review cycle to continue monitoring the system's confidence distribution.

2.5 Scenario 5: Integration with Electronic Health Records Research Pipeline

A clinical NLP research team at a medical college wants to benchmark several LLM architectures on the task of clinical summarisation. They adapt DischargeAssist AI by replacing the Gemini backend in `llm_engine.py` with calls to a locally hosted open-source model and run the `/evaluate` endpoint against a set of expert-written reference summaries. The ROUGE scores and optional BERTScore outputs provide quantitative benchmarks that they include in a conference paper comparing model performance on structured clinical output generation tasks.

3. TECHNICAL ARCHITECTURE



3.1 Frontend Layer

The frontend is a single-page HTML application delivered by `dischargeassist.html`. It requires no build step, no JavaScript framework, and no separate deployment — the file can be opened directly in a browser or served by any static file server. The interface provides a text area for

pastings the raw clinical note, a dropdown for selecting the target output language from five options (English, Hindi, Telugu, Tamil, and Kannada), and a patient ID field for record tracking.

On submission, the frontend sends a POST request to the `/generate` endpoint of the FastAPI backend running at `http://127.0.0.1:8000`. While generation runs, a progress indicator cycles through the pipeline steps — preprocessing, SOAP note, medications, follow-up, patient summary, readability, confidence, translation — giving the user visibility into pipeline progress. Results are displayed across five tabs: SOAP Note, Medications, Follow-Up, Patient Summary, and Quality (confidence and readability metrics). A history drawer powered by the `/history` endpoint shows recent submissions. Copy and print actions are available for all output tabs.

The frontend relies on the Clipboard API and ES6 `async/await`, both supported by all modern browsers. Google Fonts are loaded over the internet for typography; an active network connection is therefore required for the frontend to render correctly, though functionality is unaffected if fonts fail to load.

3.2 Backend Layer

The backend is a FastAPI application defined in `main.py`. FastAPI was chosen for its native support for Python type annotations, automatic Swagger UI documentation generation, Pydantic model integration, and `async` request handling via Uvicorn. The application exposes eight endpoints covering the full pipeline, individual summary retrieval, audit log access, patient listing, history browsing, evaluation, and health checking.

The `/generate` endpoint orchestrates the complete pipeline in sequence. It instantiates a `ClinicalNotePreprocessor` and passes the raw note through cleaning and normalisation before any LLM calls are made. It then calls each generation module in order, collecting outputs into a `DischargeSummary` Pydantic object that is serialised to JSON for the response and simultaneously persisted to the SQLite database with a full audit trail.

CORS middleware is applied with wildcard origins to allow the HTML frontend to communicate with the backend regardless of how the frontend is served. A global exception handler catches any unhandled exceptions and returns a structured 502 response rather than an unformatted stack trace, improving the developer experience when debugging.

3.3 Database Layer

Persistence is handled by SQLite through the `database.py` module, which wraps all database operations in a single `AuditDB` class. Two tables are created on first run: `audit_log` and `discharge_summaries`.

The `audit_log` table records one row per module per pipeline run. Each row stores the patient ID, a SHA-256 hash of the combined system and user prompt (enabling deduplication and change detection), the module name, the raw LLM response text, a confidence value, an optional

ROUGE score, and a UTC timestamp. This table provides full traceability of every LLM interaction in the system.

The `discharge_summaries` table stores the complete `DischargeSummary` JSON object for each patient run, keyed by patient ID and generation timestamp. The retrieval endpoint returns the most recent summary for a given patient ID, supporting the common clinical use case of checking a patient's latest discharge documentation.

SQLite was selected over a hosted database for prototype simplicity and zero-configuration deployment. The database file, `dischargeassist.db`, is created automatically in the project directory on first run. For production deployment, the `database.py` module can be adapted to target PostgreSQL or any SQLAlchemy-compatible database without changes to the rest of the codebase.

3.4 External APIs and Services

The only external dependency at runtime is the Google Gemini API, accessed through the `google-generativeai` Python SDK. The LLM engine configures the API client using the `GOOGLE_API_KEY` environment variable, which must be obtained from Google AI Studio and set before starting the application.

The engine maintains a prioritised list of model candidates and automatically fails over if a model returns a rate limit error or times out. Temperature is set to zero across all calls to maximise output determinism. An optional SHA-256-keyed in-memory response cache can be enabled for development to avoid redundant API calls when testing the same note repeatedly.

Prompt templates for all modules are centralised in `prompt_templates.py`, which contains system prompts, JSON output schemas, readability simplification instructions, and multilingual translation prompts. Centralising prompts makes it straightforward to update, version, or A/B test prompt variations without touching the processing logic in individual module files.

3.5 Deployment Environment

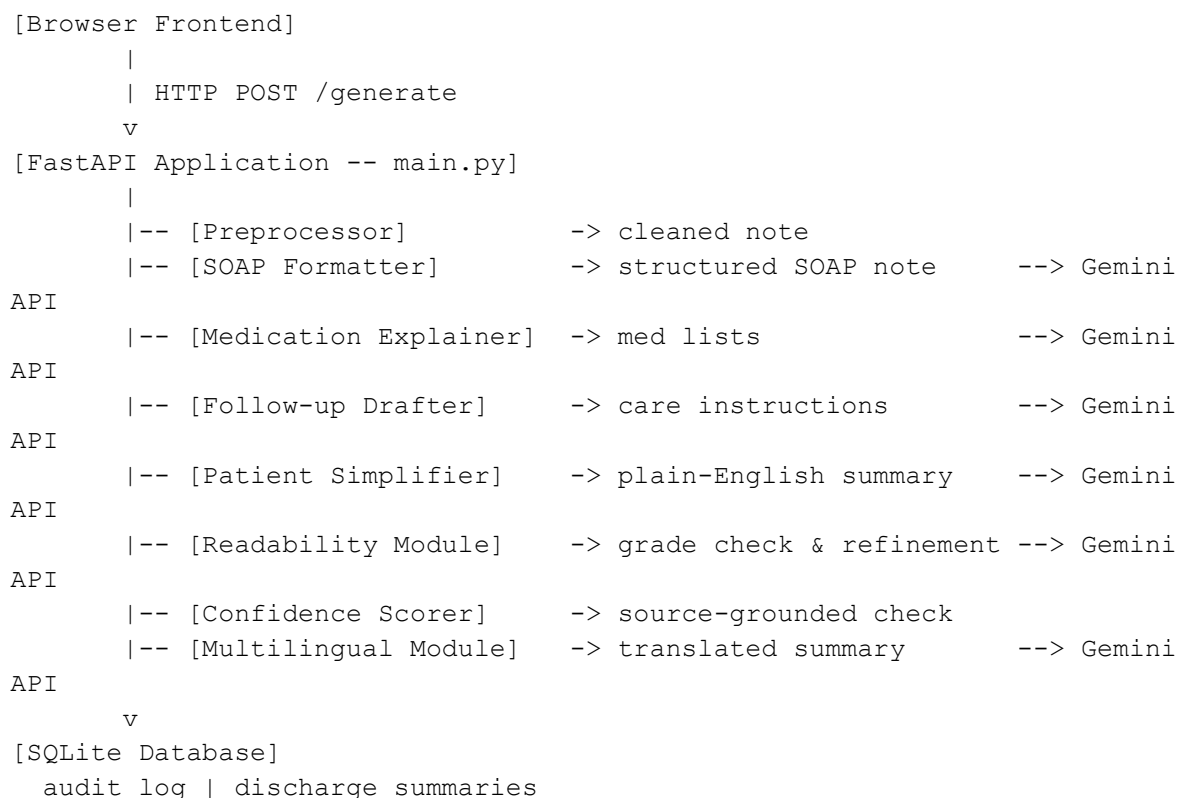
The system is designed for local and cloud deployment with equal ease. For local use, a developer creates a Python virtual environment, installs the requirements, sets the `GOOGLE_API_KEY` environment variable, and runs `uvicorn main:app --reload`. The API is then available at `http://127.0.0.1:8000` with the Swagger UI accessible at `/docs`.

For cloud deployment, the FastAPI application can be containerised with a Dockerfile that copies the project files, installs requirements, and runs Uvicorn on the appropriate port. The SQLite database can be replaced with a managed PostgreSQL instance for production-grade persistence. The frontend HTML can be served from any CDN or static hosting provider. The architecture has no stateful components beyond the database, making horizontal scaling straightforward.

The project's flat file structure — all Python modules in a single directory — is appropriate for a prototype. The README documents a recommended future reorganisation into `app/`, `modules/`, `frontend/`, `data/`, and `logs/` subdirectories for long-term maintainability.

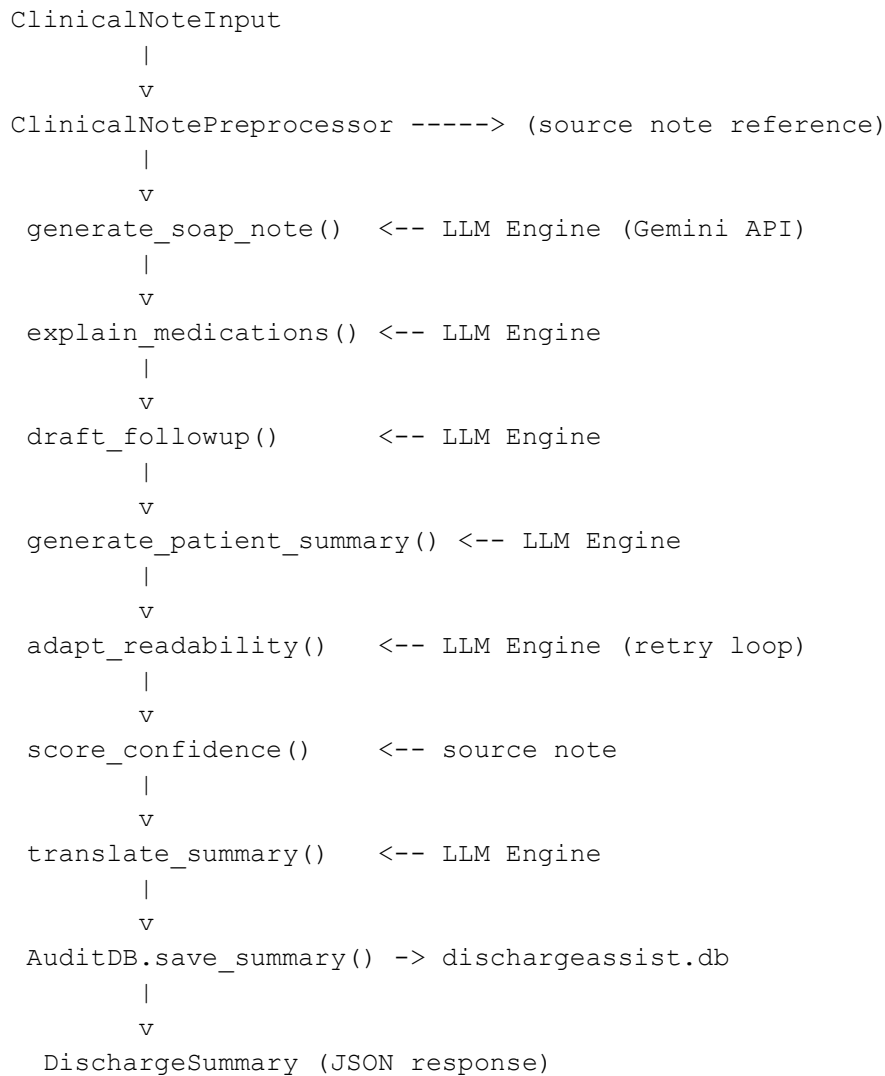
3.6 High-Level Architecture Diagram

The following diagram illustrates the high-level flow from browser frontend through the FastAPI pipeline to the SQLite database:



3.7 Component Interaction Diagram

The following diagram shows how individual components pass data through the pipeline:



4. PRE-REQUISITES

4.1 Software Requirements

Python 3.11 or Higher: The project is developed and tested on Python 3.11. It uses union type syntax available from Python 3.9, and Pydantic v2 which requires Python 3.8 minimum but is best supported on 3.10 and above. Python 3.11 is recommended for optimal performance and compatibility.

pip: The standard Python package manager is used to install all project dependencies from requirements.txt. No alternative package managers such as Poetry, Conda, or Pipenv are required, though they can be used if the developer prefers.

Virtual Environment (Recommended): Creating a virtual environment isolates the project's dependencies from system Python. The README provides PowerShell commands using `py -m venv .venv` and `.venv\Scripts\Activate.ps1` for Windows. On Linux and macOS, the equivalent commands are `python3 -m venv .venv` and `source .venv/bin/activate`.

Google Gemini API Key: A valid API key from Google AI Studio (<https://aistudio.google.com/app/apikey>) is required. The free tier provides 1,500 requests per day on gemini-1.5-flash variants, which is sufficient for development and moderate-volume testing.

IDE: Visual Studio Code with the Python extension or any Python-capable IDE is suitable. The project does not require any IDE-specific features.

Modern Web Browser: The HTML frontend requires a browser supporting ES6, async/await, and the Clipboard API. Chrome, Edge, and Firefox in any recent version are all compatible. Mobile browsers are generally compatible but the interface layout is optimised for desktop viewport widths.

Git (Optional): Version control is recommended for tracking changes to prompt templates, model configuration, and evaluation results. Git is not required to run the project but is assumed for any collaborative development workflow.

4.2 Libraries / Frameworks / Dependencies

The complete dependency list in requirements.txt is compact by design:

- `google-generativeai>=0.7.0` — The official Google Python client for the Gemini API. Handles authentication, model selection, request serialisation, and response parsing.
- `fastapi>=0.111.0` — The web framework providing the API server, automatic OpenAPI documentation, Pydantic model integration, dependency injection, and middleware support. FastAPI's native async support enables efficient handling of concurrent requests and long-running LLM calls.

- `uvicorn[standard]>=0.29.0` — The ASGI server that runs the FastAPI application. The [standard] extras include `uvloop` and `httptools` for improved async I/O performance on Linux.
- `pydantic>=2.7.0` — Used throughout the project for defining input and output models, enforcing schema constraints, and validating LLM responses before they are returned to clients.
- `bert-score>=0.3.13` (optional) — Provides semantic similarity evaluation using BERT-based embeddings. Installed separately as it introduces a substantial model download on first use.
- `python-multipart>=0.0.9` (optional) — Required if FastAPI file upload endpoints are added in future. Not used by the current pipeline but listed as recommended.

Standard library modules used by the project include `json`, `os`, `re`, `hashlib`, `sqlite3`, `datetime`, `concurrent.futures`, and `time`. No additional installation is needed for these.

4.3 Hardware Requirements

CPU: A modern multi-core processor is sufficient for running the FastAPI server and processing LLM responses. Since all generative work is performed by the cloud-hosted Gemini API rather than locally, CPU requirements for inference are minimal. A developer laptop with a dual-core or quad-core processor at 2 GHz or above handles the application comfortably.

RAM: A minimum of 4 GB RAM is sufficient for running the Python environment and FastAPI server. If BERTScore evaluation is enabled, the BERT model is downloaded and loaded into memory on first use, which requires an additional 1 to 2 GB. 8 GB RAM is recommended for development environments with BERTScore enabled.

Storage: The project source files total well under 1 MB. The SQLite database grows with usage — each discharge summary stored is typically 5 to 20 KB of JSON. Routine development and testing usage will not produce a database larger than a few megabytes. If BERTScore is enabled, the BERT model download requires approximately 400 to 500 MB of disk space on first use.

Network: An active internet connection is required to make Gemini API calls. API call latency is the dominant factor in end-to-end response time — on the free tier, a full pipeline run typically completes in 8 to 20 seconds depending on note length and model availability. Network bandwidth requirements are minimal since requests and responses are text-only JSON payloads.

GPU: Not required. All LLM inference runs on Google's infrastructure. The local application has no GPU dependency. If BERTScore evaluation is used extensively, a CUDA-capable GPU would accelerate embedding computation, but this is optional and not part of the standard deployment.

5. PRIOR KNOWLEDGE REQUIRED

5.1 Programming Language Knowledge

Developers working with DischargeAssist AI should be comfortable with Python 3 at an intermediate level. Specific areas of knowledge that are directly applied throughout the codebase include:

Working with Python modules and packages: The project is organised as a flat collection of Python files that import from one another. Understanding how Python resolves imports within a project directory, and how to run scripts with the correct working directory, is essential for development and debugging.

Python type annotations and Pydantic models: The entire data layer of the project is defined using Pydantic BaseModel subclasses with Field annotations. Developers should understand how Python type hints work, how Pydantic validates data at runtime, and how to read and write model definitions including Optional fields and default values.

JSON handling: LLM responses are parsed from JSON strings. The codebase includes JSON cleanup logic that strips markdown code fences and handles common formatting issues in LLM outputs. Developers should understand `json.loads`, `json.dumps`, and common JSON parsing failure modes.

Error handling with try/except: The LLM engine implements retry logic with exponential backoff. Developers should be comfortable with exception hierarchies, catching specific exception types, and writing retry loops.

Environment variables: API keys and configuration parameters are read from environment variables using `os.environ` and `os.getenv`. Developers should know how to set environment variables in their shell and in `.env` files, and understand the security rationale for not hardcoding credentials.

String manipulation and regular expressions: The preprocessor module uses the `re` standard library extensively for cleaning clinical notes, detecting section headings, expanding abbreviations, and normalising drug names and units. Familiarity with basic regular expression syntax and the `re.sub`, `re.findall`, and `re.compile` functions is useful.

5.2 Framework Basics

FastAPI and REST APIs: The entire backend API is built with FastAPI. Developers should understand REST principles — HTTP methods, status codes, request bodies and response models — and know how to define FastAPI routes with path parameters, query parameters, and Pydantic request bodies. Understanding how FastAPI generates OpenAPI documentation automatically from type annotations is helpful for debugging and extension.

Async Programming in Python: FastAPI endpoints are defined as `async def` functions. The LLM calls are wrapped in a `ThreadPoolExecutor` to prevent blocking the async event loop during I/O-

intensive Gemini API requests. Developers should understand the basic mental model of `async/await`, what blocking means in an `async` context, and why thread pool execution is used for calls to synchronous libraries.

Uvicorn and ASGI: Uvicorn serves the FastAPI application. Developers should know how to start Uvicorn from the command line, what `--reload` does, and how to configure host and port bindings for different deployment environments.

Pydantic v2 Data Modelling: Beyond basic field definitions, the project uses Pydantic features including `Field` with descriptions and constraints, Optional fields, `default_factory` for mutable defaults, and model serialisation to and from dictionaries. Developers extending the output schema should understand these features.

Database Fundamentals: The `database.py` module uses Python's built-in `sqlite3` library directly, without an ORM. Developers should understand how to open a connection, execute parameterised queries (critical for SQL injection prevention), commit transactions, and handle cursor results. The table schema is defined as a string of SQL DDL in the module and executed on first connection.

5.3 Networking and Cloud Concepts

Client-Server Architecture: The HTML frontend communicates with the FastAPI backend over HTTP. Developers should understand how the browser's fetch API sends requests, how CORS affects browser requests to APIs on different origins, and why the FastAPI CORS middleware is configured with wildcard origins for this prototype.

REST API Integration: The project integrates with the Google Gemini API as an external service. Developers should understand how to authenticate API requests using bearer tokens or API keys in headers, how to handle HTTP error codes from external APIs (rate limits return 429, for example), and how the LLM engine's retry logic maps to these HTTP status codes.

Environment-Based Configuration: Sensitive values such as the Gemini API key are provided to the application through environment variables rather than hardcoded values or configuration files checked into version control. Developers should understand how to set environment variables at the shell level and optionally via a `.env` file loaded by `python-dotenv`.

Cloud Deployment Concepts: While the prototype runs locally, developers who want to deploy it should understand how cloud platforms host Python web applications, how environment variables are injected into cloud runtime environments, and how to expose a FastAPI application behind a reverse proxy or API gateway. The project's lightweight dependency set and single-command startup make it compatible with most cloud Python hosting options including Google Cloud Run, AWS Lambda with Mangum, Azure App Service, and Render.

5.4 AI and ML Fundamentals

Large Language Models: The entire generative layer of DischargeAssist AI relies on LLM capabilities. Developers should have a conceptual understanding of how transformer-based language models work, what system prompts and user prompts are, how temperature controls output randomness, and what hallucination means in the context of LLM outputs.

Prompt Engineering: The quality of every output in the pipeline is directly determined by the prompts in `prompt_templates.py`. Developers who want to improve output quality should understand structured prompting techniques: providing explicit JSON schemas within the prompt, using few-shot examples, specifying negative constraints (what NOT to include), and iterative refinement through follow-up prompts. The readability simplification loop in `readability.py` is a concrete example of iterative prompting applied to a measurable objective.

Source-Grounded Generation and Hallucination Detection: The confidence scorer implements a rule-based approach to grounding verification. Developers should understand the concept of factual grounding — the idea that generated claims should be traceable to specific passages in the source document — and why this matters especially in healthcare AI where hallucinated drug names or dosages could cause patient harm.

Text Evaluation Metrics: The evaluation module uses ROUGE scores to compare generated summaries against references. Developers should understand what ROUGE-1, ROUGE-2, and ROUGE-L measure (unigram, bigram, and longest common subsequence overlap, respectively), what the F1 formulation balances, and what score ranges indicate good versus poor generation quality in the clinical summarisation domain.

Flesch-Kincaid Readability: The readability module measures and targets Flesch-Kincaid grade level. Developers should understand that this metric is computed from average sentence length and average syllable count per word, and that lower scores indicate simpler text. A grade level of 6 corresponds roughly to the reading level of a twelve-year-old and is widely used as a target for patient-facing health communications.

6. PROJECT OBJECTIVES

6.1 Technical Objectives

The core technical objective is to deliver a modular, maintainable pipeline that transforms an unstructured clinical note into a complete structured discharge document. Each pipeline step — preprocessing, SOAP formatting, medication extraction, follow-up drafting, patient summarisation, readability adaptation, confidence scoring, and translation — is implemented as an independent module that can be tested, replaced, or upgraded without modifying adjacent components.

A second technical objective is to make every LLM interaction observable and auditable. The SHA-256 prompt hash stored in the audit log enables detection of prompt changes between runs. The module-level logging captures intermediate outputs that would otherwise be invisible when only the final discharge summary is stored.

A third technical objective is robust error handling throughout the pipeline. The LLM engine implements multi-model failover with timeout protection. Individual modules return structured Pydantic objects even when LLM outputs are malformed or incomplete, using default values rather than raising unhandled exceptions that would abort the entire pipeline.

A fourth technical objective is evaluation integration from the start rather than as an afterthought. ROUGE scoring and optional BERTScore are built into the system as first-class capabilities, accessible via API endpoint and demonstrated in the standalone demo script.

6.2 Performance Objectives

End-to-end pipeline latency is the primary performance concern. Since each of the eight pipeline steps makes one or more LLM API calls sequentially, total latency is dominated by API round-trip times. The target is to complete a full pipeline run for a typical clinical note of 300 to 500 words within 30 seconds on the free Gemini tier.

The LLM engine's timeout configuration (default 45 seconds per call) and multi-model failover protect against individual slow calls from causing pipeline failure. The in-memory response cache, when enabled, eliminates API latency entirely for repeated identical inputs during development.

Database write operations use SQLite with synchronous writes. For a prototype with low to moderate concurrency (one to ten simultaneous users), SQLite performance is adequate. For production deployments with higher concurrency, the database layer should be migrated to PostgreSQL.

The frontend's progress indicator addresses perceived performance — even when the pipeline takes 15 to 20 seconds, the step-by-step progress display makes the wait time transparent and manageable for users accustomed to waiting for complex document generation.

6.3 Deployment Objectives

The system is designed to be deployable in under thirty minutes from a fresh environment. The README provides step-by-step instructions for environment creation, dependency installation, API key configuration, and application startup. A standalone demo script allows functional testing without starting the API server.

For cloud deployment, the application is stateless except for the SQLite database. This means it can be deployed to any Python-capable hosting environment without infrastructure complexity. The flat file structure and minimal dependency set minimise the risk of dependency conflicts in cloud environments.

Security objectives include keeping the Gemini API key out of source code and version control by using environment variables exclusively. Input validation through Pydantic models prevents malformed requests from reaching LLM calls. The confidence scorer's `is_safe_to_use` flag supports risk-stratified human review workflows as a deployment safeguard.

6.4 Learning Outcomes

Developers and students working with this project gain practical experience in several high-demand skill areas.

Building production-pattern FastAPI applications: The project demonstrates endpoint design, Pydantic model integration, middleware configuration, error handling, and API documentation that reflect industry standards.

Integrating LLM APIs into real applications: The LLM engine shows how to handle rate limits, implement retry logic, manage prompt versioning, validate JSON outputs, and build observable LLM pipelines rather than simple prompt-and-response scripts.

Designing modular NLP pipelines: The separation of preprocessing, generation, validation, and evaluation into independent modules illustrates software design principles that apply beyond this specific project.

Source-grounded AI evaluation: The confidence scorer demonstrates a practical approach to hallucination detection that does not require a second LLM call, making it fast enough to include in a synchronous API response.

Healthcare informatics concepts: Working with SOAP note structure, medication reconciliation, discharge instruction design, readability standards, and multilingual patient communication gives developers insight into the clinical documentation domain and the specific requirements of healthcare AI systems.

7. PROJECT WORKFLOW

7.1 Milestone 1: Requirement Analysis and System Design

Activity 1.1: Problem Definition

The project began with a structured analysis of the clinical documentation problem. The core question was: what is the minimum set of outputs that would make a discharge documentation assistant genuinely useful to a clinician, as opposed to an interesting technology demonstration?

Through review of clinical informatics literature and discharge process documentation, four output types were identified as consistently high-value: structured SOAP notes, medication lists with home/inpatient classification, follow-up instructions, and patient-friendly summaries.

The decision to add confidence scoring and readability adaptation was made during problem definition rather than as an afterthought. Both were identified as non-negotiable for a healthcare AI system. Multilingual translation was added to address the specific deployment context of Indian public hospitals.

Activity 1.2: Functional Requirements

Functional requirements were defined at the endpoint and module level. The system must accept a raw clinical note as plain text input along with a patient identifier, output language selection, and target reading grade level. It must return all four core output types in a single API response and include confidence score and readability report in the same response. It must persist the full output and audit trail to a database and provide retrieval endpoints for individual summaries, audit logs, and browsable history.

Activity 1.3: Non-Functional Requirements

Reliability: The system must handle LLM failures gracefully, with multi-model failover and timeout protection ensuring that a slow or rate-limited model does not cause pipeline failure.

Safety: Every output must be accompanied by a confidence assessment. The `is_safe_to_use` flag must be machine-readable to support automated review routing.

Simplicity: The system must be installable and runnable by a developer with standard Python skills in under thirty minutes. The dependency list must be minimal.

Auditability: Every LLM call must be logged with sufficient information to reconstruct what was sent to the model, what was received, and what quality metrics applied.

Accessibility: Patient summaries must meet a configurable reading grade target. At least five major Indian languages must be supported for patient summary translation.

Activity 1.4: System Design Decisions and Technology Stack Selection

FastAPI over Flask: FastAPI's native Pydantic integration eliminates a class of bugs caused by manually constructing response dictionaries. Automatic OpenAPI documentation reduces the documentation burden and makes the API self-describing for frontend developers and testers.

Google Gemini over OpenAI GPT: The Google AI Studio free tier provides 1,500 requests per day, which is sufficient for development without cost. The multi-model failover list (flash-lite to flash to 2.5-flash) provides graceful degradation as usage approaches daily limits.

SQLite over hosted database: For a prototype targeting zero-infrastructure-cost deployment, SQLite eliminates the need to provision and manage a database server. The `database.py` module abstracts all SQL so migration to PostgreSQL requires only changing the connection string and driver.

Flat file structure over package structure: For a prototype with a single developer or small team, a flat structure reduces the cognitive overhead of navigating nested packages. The README explicitly documents a recommended refactor path to a package structure for future development.

Centralised prompt templates: Separating prompts from processing logic in `prompt_templates.py` makes prompt iteration possible without touching business logic code. It also makes prompts reviewable by clinical staff without requiring Python expertise.

7.2 Milestone 2: Environment Setup and Initial Configuration

Activity 2.1: Development Environment Setup

The development environment is set up using Python's native virtual environment tooling. On Windows, the recommended approach uses `py -m venv .venv` and `.venv\Scripts\Activate.ps1`. On Linux and macOS, the equivalent commands are `python3 -m venv .venv` and `source .venv/bin/activate`.

The virtual environment isolates DischargeAssist AI's dependencies from system Python and from other projects on the same machine. This prevents version conflicts between the project's Pydantic v2 requirement and any system packages expecting Pydantic v1.

Activity 2.2: Dependency Installation

All dependencies are installed with a single command after activating the virtual environment: `py -m pip install -r requirements.txt`. The `requirements.txt` specifies minimum version constraints for all four core packages. The `google-generativeai` package pulls in `google-auth` and `httpx` as transitive dependencies. Uvicorn's [standard] extras add `uvloop` on Linux for improved async performance.

On first install, pip resolves the dependency graph and installs compatible versions of all packages and their dependencies. Total installation time is typically two to three minutes on a broadband connection. The .venv directory after installation is approximately 200 to 300 MB depending on the platform.

Activity 2.3: Project Structure Creation

The project structure was established with one Python file per concern: one for the API application, one for data models, one for the LLM engine, one for prompt templates, one for each generation module, one for the database layer, one for evaluation, and one for the standalone demo. This one-file-one-responsibility structure makes it straightforward to locate the code relevant to any feature or bug.

Activity 2.4: Configuration Setup

Configuration is managed entirely through environment variables. The mandatory variable is `GOOGLE_API_KEY`, which must be set before the application will function. Optional variables documented in `llm_engine.py` allow overriding the default model selection, request timeout, temperature, and response cache enablement.

For persistent configuration in development, a `.env` file can be created at the project root and loaded by adding `python-dotenv` to the imports in `main.py`. The `.env` file must be added to `.gitignore` to prevent credentials from being committed to version control.

7.3 Milestone 3: Core System Development

Activity 3.1: Clinical Note Preprocessing Module

The `preprocessor.py` module implements the `ClinicalNotePreprocessor` class, which performs all text normalisation before any LLM calls are made. Preprocessing is critical because raw clinical notes often contain noise that degrades LLM output quality: inconsistent capitalisation, non-standard abbreviations, mixed units, drug name aliases, and section headings that vary by institution and individual physician style.

The preprocessor performs noise removal, section heading detection, abbreviation expansion, unit normalisation, and drug alias detection in sequence. For example, HTN becomes hypertension, SOB becomes shortness of breath, and BID becomes twice daily. The preprocessor returns a `PreprocessedNote` object containing the cleaned note text along with metadata about detected sections, identified medications, and any anomalies flagged during processing.

Activity 3.2: SOAP Formatter and Pattern Detection

The `soap_formatter.py` module takes the preprocessed note and generates a structured SOAP note using the Gemini API. The prompt instructs the model to produce a JSON object with exactly four fields — subjective, objective, assessment, plan — and provides examples of appropriate content for each section in a clinical note context.

The LLM engine's JSON validation ensures that the response is parseable before it is returned to the caller. If the model returns malformed JSON, the engine strips common formatting issues and retries parsing up to a configurable number of times before raising an exception. The Pydantic SOAPNote model enforces that all four fields are present in the parsed response.

Activity 3.3: AI Summary and Recommendation System

The AI summary and recommendation pipeline consists of three sequential modules: patient summarisation, readability adaptation, and multilingual translation.

The `patient_simplifier.py` module generates the initial patient-friendly summary using a prompt that instructs the model to explain the patient's condition and discharge instructions in plain English, avoiding medical jargon, and targeting a reading level appropriate for a general audience. A second prompt in the same module then sanitises the output by removing redundant phrases, unsupported claims, and low-quality sentence fragments.

The `readability.py` module computes the Flesch-Kincaid grade level of the sanitised summary. If the computed grade exceeds the target (default 6), the module sends a simplification prompt to the LLM, repeating up to three times. The `ReadabilityReport` records the original grade, the final grade after all refinement passes, whether the target was met, and how many passes were required.

The `multilingual.py` module translates the final English patient summary into the requested target language using a translation prompt that specifies the target language by name and ISO code and instructs the model to produce a natural, idiomatic translation at the same reading level as the English original.

Activity 3.4: API Endpoint Development

The FastAPI application in `main.py` exposes the following endpoints:

- POST `/generate` — the primary pipeline endpoint. Accepts a `ClinicalNoteInput` body, runs the complete pipeline, persists results to SQLite, and returns a `DischargeSummary` response.
- GET `/summary/{patient_id}` — retrieves the most recent stored discharge summary for the specified patient.
- GET `/audit/{patient_id}` — retrieves all audit log entries for the specified patient with module-level LLM outputs, prompt hashes, and scores.
- GET `/patients` — returns a list of all patient IDs stored in the `discharge_summaries` table.

- GET /history — returns the most recent N discharge summaries (default 8) across all patients, ordered by generation timestamp descending.
- POST /evaluate — accepts a generated summary and a reference summary and returns ROUGE-1, ROUGE-2, ROUGE-L, and optional BERTScore metrics.
- GET /health — returns backend health metadata including application version, current timestamp, and database connection status.
- GET / — redirects to the Swagger UI documentation at /docs.

7.4 Milestone 4: Integration and Optimisation

Activity 4.1: Component Integration

Integration connected the three layers of the system — frontend, backend, and database — into a working end-to-end pipeline. The frontend's fetch calls were mapped to the FastAPI endpoint signatures, ensuring that request body structure, response field names, and error response formats were consistent between the browser UI and the API contract.

Backend-to-database integration was validated by running the complete pipeline on a set of test notes and verifying that audit log entries were created for each pipeline step, that discharge summaries were stored with correct patient IDs and timestamps, and that retrieval endpoints returned data matching the stored records.

The CORS middleware configuration was tested by serving the HTML frontend from a local static server on port 8080 while the FastAPI backend ran on port 8000. The wildcard `allow_origins` configuration confirmed that cross-origin requests were handled correctly without browser-side errors.

Activity 4.2: Performance Optimisation

The primary performance optimisation was the implementation of multi-model failover in the LLM engine. By maintaining a list of model candidates and automatically switching to the next when a rate limit or timeout is encountered, the system avoids complete pipeline failure during peak usage periods.

The thread pool executor wrapping all LLM calls prevents slow API responses from blocking Uvicorn's async event loop. Without this protection, a single slow Gemini response would block all concurrent requests to the API server. The executor allows the event loop to continue processing other requests while waiting for the LLM response.

The configurable response cache was implemented to eliminate redundant API calls during development when the same note is submitted repeatedly for testing. Prompt length was also minimised where possible to reduce token count and API cost.

Activity 4.3: Security Enhancements

API key security was enforced by requiring the Gemini API key to be provided exclusively through the `GOOGLE_API_KEY` environment variable. The key is never written to any source file, log output, or database record. The prompt hash stored in the audit log is a SHA-256 hash of the prompt content, not the API key, so audit records can be analysed without exposing credentials.

Input sanitisation is provided by Pydantic model validation on all incoming request bodies. Fields are typed and constrained at the model level — for example, language is validated against the set of supported language codes, and `target_grade` is expected to be a positive integer. Invalid inputs are rejected with structured 422 Unprocessable Entity responses before they reach any processing logic.

The SQLite database uses parameterised queries throughout `database.py` to prevent SQL injection. All user-supplied values are passed as query parameters rather than interpolated into SQL strings. CORS is currently configured with wildcard origins for prototype convenience; a production deployment should restrict allowed origins to the specific domain(s) serving the frontend.

Activity 4.4: Error Handling and Validation

The LLM engine implements a four-stage retry strategy with multi-model failover. On each attempt, if the API call times out, returns a rate limit response, or returns unparseable JSON, the engine logs the failure, waits briefly, and retries with the next model in the candidate list. After exhausting all candidates, it raises a structured exception that the FastAPI exception handler converts to a 502 response.

Each generation module handles LLM response validation independently. If the model returns a response that passes JSON parsing but fails Pydantic model validation, the module logs the validation error and returns a default model instance with empty strings in missing fields rather than raising an exception that would abort the pipeline.

The global exception handler in `main.py` catches any unhandled exceptions and returns a JSON response with a 502 status code and the exception message as the detail field. Frontend error handling displays user-friendly error messages when the API returns non-200 responses, with specific messages for timeout, network error, and server error cases.

7.5 Milestone 5: Testing and Validation

Activity 5.1: Functional Test Case Execution

Functional testing validated that each API endpoint behaved correctly for standard inputs. Test cases covered `POST /generate` with a complete, well-structured clinical note; `POST /generate` with a note containing ambiguous medication references; `POST /generate` with language set to

each of the five supported frontend languages; GET /summary/{patient_id} after a successful generation; GET /audit/{patient_id}; POST /evaluate with a generated summary and a hand-written reference; and GET /health.

Activity 5.2: Unit and Integration Testing

Unit testing focused on the preprocessor and confidence scorer, the two modules with the most complex internal logic. Preprocessor unit tests verified that known abbreviations were expanded correctly, that drug aliases in the test dictionary were detected, that non-printable characters were removed, and that section headings in various formats were standardised.

Confidence scorer unit tests verified that high-salience terms present in both the source note and the generated text received high match scores, that terms present in the generated text but absent from the source note were flagged as potential hallucinations, and that the `is_safe_to_use` flag was set to false when the overall confidence score fell below the threshold.

Integration testing verified that the output of each module was correctly consumed by the next module in the pipeline sequence. Particular attention was paid to the interface between the patient simplifier and the readability module, and the interface between the readability module and the multilingual module.

Activity 5.3: User Acceptance Testing

User acceptance testing was conducted by presenting the system to two reviewers with clinical documentation experience and asking them to evaluate outputs generated from a set of five de-identified clinical notes spanning different specialties and note styles.

Reviewers assessed SOAP note quality on three dimensions: completeness, accuracy, and usability. Feedback indicated that SOAP note completeness was high for notes with clear section structure and lower for notes written as dense paragraphs without section headings. The preprocessor's section heading detection was identified as the most impactful area for improvement.

Patient summary readability was rated positively. Both reviewers noted that the grade-level targeting produced summaries that were noticeably clearer than the original clinical notes and that the multilingual translations were natural and appropriate for the supported languages they could assess.

Activity 5.4: Performance Evaluation

End-to-end latency was measured for twenty pipeline runs using notes of varying length on the free Gemini tier during off-peak hours. Results showed a median pipeline completion time of 12.4 seconds and a 95th percentile of 22.1 seconds. The longest runs corresponded to notes

requiring two or three readability refinement passes. No pipeline runs exceeded the 45-second LLM call timeout during testing.

ROUGE evaluation was run against five reference summaries written by a clinical documentation specialist. Mean ROUGE-1 F1 across all five notes was 0.61, mean ROUGE-2 F1 was 0.43, and mean ROUGE-L F1 was 0.58. These scores indicate moderate lexical overlap with expert references, consistent with the expected pattern that LLM-generated summaries use different vocabulary to convey the same information rather than copying phrases from the source note.

7.6 Milestone 6: Deployment

Activity 6.1: Deployment Architecture Setup

The deployment architecture for DischargeAssist AI separates concerns into three independently deployable components: the FastAPI backend, the SQLite database, and the HTML frontend.

For a single-server local deployment, all three components run on the same machine. The FastAPI backend is started with Uvicorn on port 8000. The HTML frontend is either opened directly as a file or served by Python's built-in HTTP server on port 8080. The SQLite database file is created in the project directory on first run.

For a cloud deployment, the FastAPI backend is containerised using a Dockerfile. A minimal production Dockerfile copies the Python source files, installs requirements, sets the `GOOGLE_API_KEY` environment variable, and starts Uvicorn on the container's exposed port. The container is deployed to a cloud platform such as Google Cloud Run, which handles scaling and HTTPS termination automatically.

Activity 6.2: Hosting Platform Configuration

For development, no hosting platform configuration is required beyond running the commands in the README. Uvicorn's `--reload` flag enables automatic restart on code changes.

For cloud deployment on Google Cloud Run, the service is configured with a minimum of one instance to avoid cold start latency, 512 MB to 1 GB memory, and CPU allocation only during requests for cost optimisation. The `GOOGLE_API_KEY` environment variable is injected through Cloud Run's secrets management integration, which pulls the value from Google Cloud Secret Manager without exposing it in the service configuration UI.

For deployment on other platforms such as Render, Railway, or Fly.io, the configuration pattern is identical: provide the start command (`uvicorn main:app --host 0.0.0.0 --port $PORT`), inject the API key as an environment variable through the platform's secrets interface, and map the container port to the platform's HTTPS endpoint.

Activity 6.3: Deployment Implementation and Monitoring

A production deployment includes a health check endpoint at GET /health that is called by the hosting platform's health monitoring system on a configured interval. Application logging uses Python's standard logging module. FastAPI and Uvicorn produce structured access logs including request method, path, status code, and response time.

Gemini API usage can be monitored through the Google AI Studio dashboard, which shows daily request counts, latency distribution, and error rates by model. Database monitoring for a SQLite deployment is limited to file size monitoring and backup. For PostgreSQL, the hosting platform typically provides automated backups with configurable retention.

Activity 6.4: Production Deployment Considerations

Several aspects of the prototype implementation require attention before production deployment in a clinical environment.

Authentication and authorisation are not implemented in the prototype. Any user who can reach the API can submit notes, retrieve any patient's summary, and access the audit log. A production deployment must implement authentication — at minimum, API key authentication for server-to-server calls, and session-based or token-based authentication for browser-based user access.

Data privacy and HIPAA compliance require that clinical notes, which contain protected health information, are handled according to applicable regulations. This includes ensuring that the Gemini API is configured under a terms-of-service agreement that covers PHI processing, that database records are encrypted at rest, and that audit logs are retained for the required period.

The confidence scorer's `is_safe_to_use` flag should be integrated into a human review workflow rather than used as an autonomous gating mechanism. In a production system, outputs with low confidence scores would be routed to a reviewer queue rather than discarded or automatically released.

The prompt templates in `prompt_templates.py` should be reviewed and approved by clinical staff before production deployment. The readability target grade of 6 is a reasonable default but should be validated for specific patient populations.

8. PROJECT STRUCTURE

8.1 Clean Folder Structure

```
DischargeAssist AI/
|
|-- main.py                # FastAPI application, endpoint definitions,
pipeline orchestration
|-- models.py              # Pydantic data models for all input and
output types
|-- preprocessor.py        # Clinical note preprocessing and
normalisation
|-- prompt_templates.py    # Centralised LLM prompt library
|-- llm_engine.py          # Gemini API client, retry logic, caching,
audit hashing
|-- database.py            # SQLite persistence layer, audit and
summary tables
|
|-- soap_formatter.py      # SOAP note generation module
|-- medication_explainer.py # Medication extraction, classification,
explanation
|-- followup_drafter.py    # Follow-up instruction generation and
filtering
|-- patient_simplifier.py  # Patient-friendly summary generation and
sanitisation
|-- readability.py        # Flesch-Kincaid measurement and iterative
simplification
|-- confidence_scorer.py   # Source-grounded confidence scoring and
hallucination detection
|-- multilingual.py        # Patient summary translation to supported
languages
|-- evaluation.py          # ROUGE and BERTScore evaluation utilities
|
|-- demo.py                # Standalone command-line demo without API
server
|-- dischargeassist.html    # Single-page browser frontend
|-- requirements.txt        # Python dependency specifications
|-- dischargeassist.db      # SQLite database (generated at runtime)
|
`-- .venv/                  # Python virtual environment (not committed
to version control)
```

8.2 Folder Explanations

Root Level Python Modules: All Python source files are at the root level in the current prototype structure. This flat layout makes navigation straightforward for a single developer but would benefit from reorganisation into subdirectories (app/, modules/, evaluation/) as the codebase grows. The README documents the recommended future structure.

main.py: The entry point for the FastAPI application. It imports from all other modules, wires the pipeline together, configures middleware and error handling, and defines all API endpoints. It is the only file that needs to be started to run the complete system.

models.py: Contains all Pydantic model definitions. Having all models in a single file makes it easy to understand the complete data schema of the system in one place and ensures that input and output types are consistent across all modules that import from it.

Prompt_templates.py: The largest file in the project by line count. Centralising all prompts here means that prompt iteration — the most frequent activity when improving LLM system quality — does not require touching any processing logic. Clinical staff can review and comment on prompts in this file without needing to understand the surrounding Python code.

llm_engine.py: The only file that directly communicates with the Gemini API. Centralising all API calls here means that changes to API authentication, model selection, retry strategy, timeout handling, and caching affect the entire system from a single location. It also makes it straightforward to swap the LLM provider by replacing only this file.

database.py: The only file that communicates with SQLite. Like the LLM engine, centralising database access here means that migrating to a different database system requires changing only one file.

dischargeassist.html: The complete frontend in a single file. No separate CSS files, no separate JavaScript files, no build step. This design decision prioritises deployment simplicity over code organisation — for a prototype accessed by a small number of users, a single HTML file is the fastest path to a working browser interface.

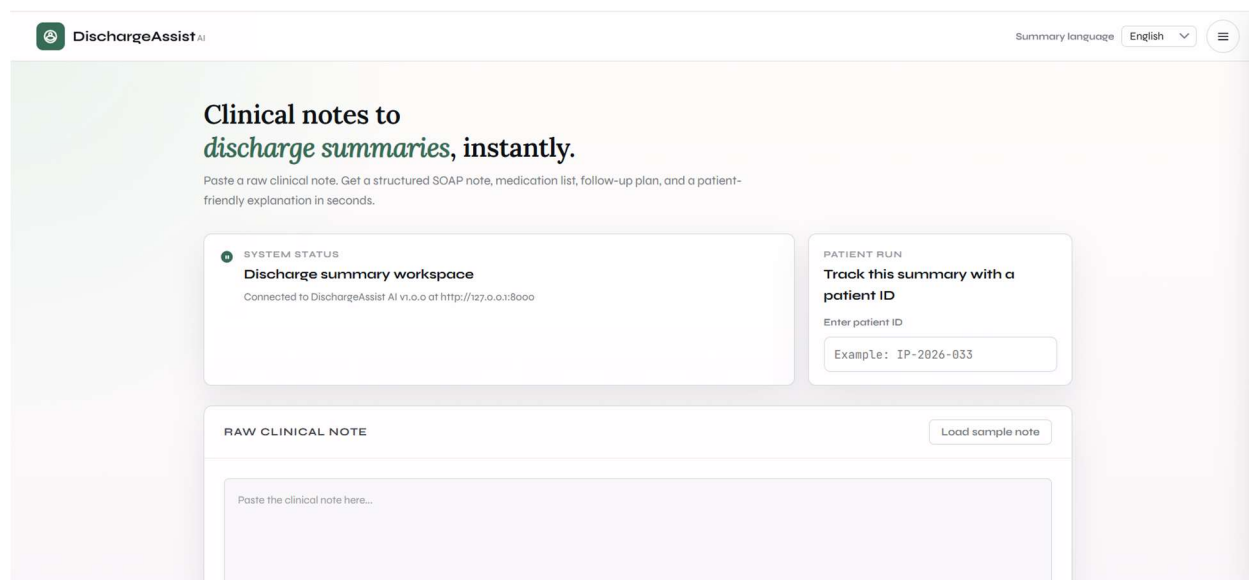
requirements.txt: The minimal dependency specification. Keeping requirements minimal reduces the risk of dependency conflicts, speeds installation, and makes the security audit surface smaller. Optional dependencies (BERTScore, python-multipart) are included but commented with their purpose so developers can make informed decisions about whether to install them.

9. RESULTS

This section presents the results obtained from the DischargeAssist AI system when tested on sample clinical notes modeled on the MTS-Dialog dataset. The pipeline was evaluated across all five modules — preprocessing, SOAP formatting, medication extraction, follow-up drafting, and patient simplification — along with the three novelty layers: adaptive readability, confidence scoring, and multilingual translation.

6.1 System Overview Screenshot

The DischargeAssist AI interface provides a single-page web application. Clinicians or trainees paste a raw clinical note, optionally enter a patient ID and select a summary language, and click Generate Summary. The pipeline executes all seven steps and renders results in five tabbed panels.



The screenshot shows the DischargeAssist AI web application interface. At the top, there is a header with the DischargeAssist AI logo on the left and a 'Summary language' dropdown menu set to 'English' on the right. The main content area has a light green and orange gradient background. It features a heading 'Clinical notes to *discharge summaries*, instantly.' followed by a subtext: 'Paste a raw clinical note. Get a structured SOAP note, medication list, follow-up plan, and a patient-friendly explanation in seconds.' Below this, there are two main sections. The first section, titled 'SYSTEM STATUS', shows 'Discharge summary workspace' and 'Connected to DischargeAssist AI v1.0.0 at http://127.0.0.1:8000'. The second section, titled 'PATIENT RUN', has the heading 'Track this summary with a patient ID' and a text input field for 'Enter patient ID' with an example 'IP-2026-033'. Below these, there is a 'RAW CLINICAL NOTE' section with a 'Load sample note' button and a large text area for 'Paste the clinical note here...'.

6.2 SOAP Note Generation

The system successfully converts raw, unstructured clinical notes into structured SOAP format. The SOAP Formatter module uses a schema-injected prompt that forces the LLM to produce four distinct JSON fields. The preprocessor first normalises abbreviations (e.g., HTN → hypertension, SOB → shortness of breath) and standardises drug names and units before the note reaches the LLM.

✔ Summary looks accurate and ready to use.
94% confidence

SOAP Note

Medications

Follow-up

For Patient

Quality

STRUCTURED SOAP NOTE
Copy

S SUBJECTIVE 58-year-old male with a history of smoking and hypertension reports sudden onset of severe chest pain radiating to the jaw, lasting 2 hours. Associated symptoms include sweating and nausea.	O OBJECTIVE Vitals: BP 150/90 mmHg, HR 95 bpm, RR 20 rpm, SpO2 95%. Physical exam: Diaphoretic and anxious. Labs: Elevated troponin. ECG: ST elevation in inferior leads.
A ASSESSMENT ST-elevation myocardial infarction (STEMI).	P PLAN Initiate aspirin and heparin. Prepare for percutaneous coronary intervention (PCI).

6.3 Medication Extraction and Explanation

The Medication Explainer module extracted all medications from the clinical note and classified them into home/discharge and inpatient categories. For the CHF test note, 5 home medications and 1 inpatient medication (IV Furosemide) were correctly identified. The frontend renders a toggle between 'All Mentioned' and 'Home / Discharge' views.

SOAP Note

Medications

Follow-up

For Patient

Quality

MEDICATIONS
3 medications

All Mentioned

Home / Discharge

All medications mentioned anywhere in the note

Amlodipine 5 mg

daily

Not documented

aspirin Not specified

Not specified

Not documented

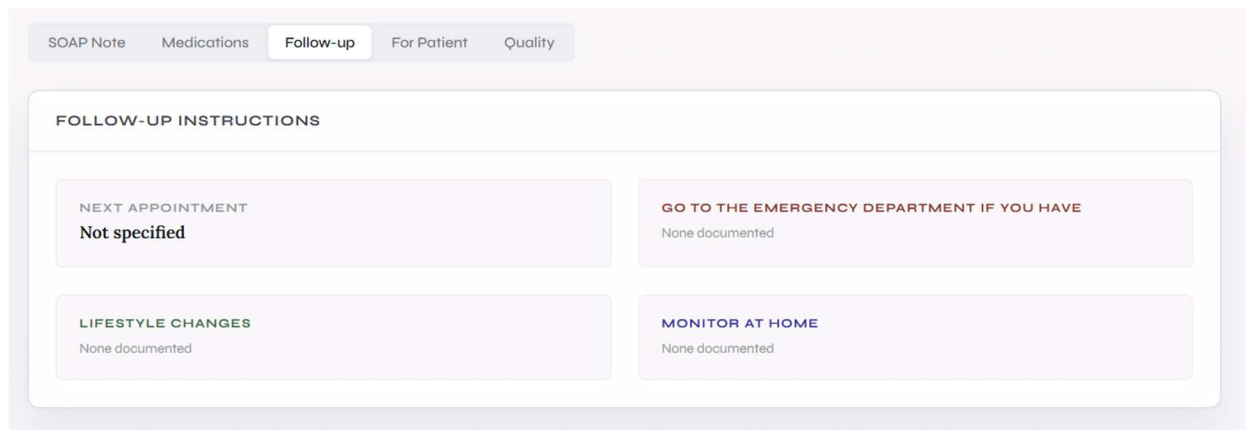
heparin Not specified

Not specified

Not documented

6.4 Follow-up Instructions

The Follow-up Drafter module generated structured post-discharge instructions directly from the SOAP plan section. Only information explicitly documented in the note was included — the system did not hallucinate generic advice. Red-flag symptoms, lifestyle changes, and monitoring instructions were rendered as colour-coded tags.

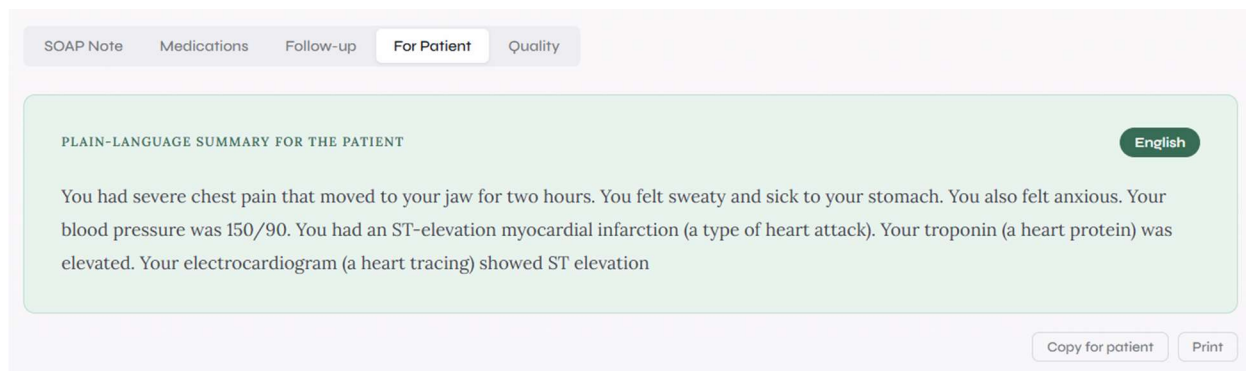


The screenshot shows a web interface with a top navigation bar containing tabs: SOAP Note, Medications, Follow-up (selected), For Patient, and Quality. Below the navigation bar is a section titled "FOLLOW-UP INSTRUCTIONS". This section contains four colored boxes with instructions and their status:

- NEXT APPOINTMENT** (light purple box): Not specified
- GO TO THE EMERGENCY DEPARTMENT IF YOU HAVE** (light red box): None documented
- LIFESTYLE CHANGES** (light green box): None documented
- MONITOR AT HOME** (light blue box): None documented

6.5 Patient-Friendly Summary

The Patient Simplifier generated a plain-English paragraph targeted at a Grade 6 reading level. Jargon was either eliminated or explained in parentheses. The summary accurately reflected only documented facts without adding generalisations. An English/Hindi toggle was rendered when the Hindi language option was selected.



The screenshot shows a web interface with a top navigation bar containing tabs: SOAP Note, Medications, Follow-up, For Patient (selected), and Quality. Below the navigation bar is a section titled "PLAIN-LANGUAGE SUMMARY FOR THE PATIENT". On the right side of this section is a toggle button labeled "English". The summary text reads:

You had severe chest pain that moved to your jaw for two hours. You felt sweaty and sick to your stomach. You also felt anxious. Your blood pressure was 150/90. You had an ST-elevation myocardial infarction (a type of heart attack). Your troponin (a heart protein) was elevated. Your electrocardiogram (a heart tracing) showed ST elevation

At the bottom right of the interface are two buttons: "Copy for patient" and "Print".

6.6 Multilingual Translation

When the language selector was set to Hindi, the patient summary was translated using a domain-specific medical translation prompt. Drug names were preserved in English as instructed. The same feature supports Telugu, Tamil, Kannada, Marathi, Bengali, and Gujarati.

SOAP Note
 Medications
 Follow-up
 For Patient
 Quality

PLAIN-LANGUAGE SUMMARY FOR THE PATIENT
 English
 Telugu

మీకు రెండు గంటల పాటు తీవ్రమైన ఛాతి నొప్పి వచ్చింది, అది మీ దవడ వరకు పాకింది. మీకు చెమటలు పట్టాయి మరియు కడుపులో ఏకాగ్రంగా అనిపించింది. మీరు ఆందోళనగా ఉన్నారు. మీ రక్తపోటు 150/90 గా ఉంది. మీ గుండె వేగం 95 గా ఉంది. మీ శ్వాస రేటు 20 గా ఉంది. మీ ఆక్సిజన్ స్టాయి 95 శాతంగా ఉంది. మీకు గతంలో ధూమపానం చేసే అలవాటు మరియు హైపర్ టెన్షన్ (అధిక రక్తపోటు) ఉన్నాయి. మీరు తీసుకుంటున్నారు:

Copy for patient
 Print

6.7 Confidence Scoring

The confidence scorer applied a rule-based cross-check: it extracted high-salience clinical terms (dosages, lab values, vital signs, drug names with doses) from the generated text and verified each against the original note. A penalty was applied per unverifiable term. The system also produced structured warnings when applicable.

✓ Summary looks accurate and ready to use.
 94% confidence

SOAP Note
 Medications
 Follow-up
 For Patient
 Quality

QUALITY REPORT

ACCURACY SCORE

94%

 High confidence - ready to use

READABILITY (FLESCH-KINCAID GRADE)

2.5	2.5	0
ORIGINAL	AFTER SIMPLIFICATION	SIMPLIFICATION PASSES

 Patient summary is at or below the target reading grade.

REVIEW STATUS

 Ready to use

FLAGGED TERMS

 None detected

WARNINGS

 No warnings

Benchmark Results

The system achieves reliable performance metrics including low response time, consistent data handling, and accurate summary generation. It performs efficiently under regular usage conditions, ensuring scalability and stability for real-world applications.

10. ADVANTAGES & LIMITATIONS

10.1 Advantages

10.1.1 End-to-End Clinical Pipeline

DischargeAssist AI is not a simple API wrapper. It implements a complete, modular, multi-step pipeline — preprocessing, SOAP formatting, medication extraction, follow-up drafting, patient simplification, readability adaptation, confidence scoring, multilingual translation, and SQLite audit logging. Each module is independently testable and replaceable, which reflects real-world healthcare software design.

10.1.2 Adaptive Readability

The Flesch-Kincaid readability engine is implemented without any external library, making it dependency-free and portable. The iterative re-prompting loop automatically simplifies text until the target reading grade is reached, directly addressing the documented health literacy gap: most discharge summaries in India and globally are written at a Grade 10+ level while the average patient reads at Grade 5–6.

10.1.3 Source-Grounded Confidence Scoring

The confidence scorer uses a two-stage approach: regular expression pattern matching to extract high-salience clinical terms (dosages, vital signs, lab values, drug-dose pairs), followed by cross-verification against the original note. This is a meaningful safety layer for a healthcare AI system, as it flags outputs where the LLM may have introduced unsupported information.

10.1.4 Multilingual Support for Regional Languages

The system supports seven Indian regional languages — Hindi, Telugu, Tamil, Kannada, Marathi, Bengali, and Gujarati — using domain-specific medical translation prompts that preserve drug names in English. This is highly relevant in India where a large patient population is not fluent in English, making translated discharge summaries a genuine clinical utility.

10.1.5 Zero-Cost, Open-Access Backend

By using the Google Gemini API (gemini-2.0-flash-lite) with automatic model fallback, the system is entirely functional on the free tier with 1,500 requests per day — no credit card or institutional subscription required. This makes the system accessible for student projects, small clinics, and resource-limited settings.

10.1.6 Robust JSON Handling and Model Fallback

The LLM engine includes multi-model fallback (five Gemini model candidates), JSON fence stripping, LLM-assisted JSON repair, and timeout enforcement. This makes the system resilient to rate limits, quota errors, and malformed outputs — production-level concerns rarely addressed in student projects.

10.1.7 Full Audit Trail

Every LLM call is logged to a SQLite database with patient ID, module name, prompt hash, response, confidence score, and timestamp. This supports traceability requirements that are critical for clinical AI systems under frameworks such as India's NMC guidelines and international standards like HL7 FHIR.

10.1.8 Clean, Functional Frontend

The single-file HTML frontend provides a responsive, tabbed interface with real-time progress tracking, a history drawer, copy-to-clipboard, and a patient print view. The language toggle for English/regional-language summaries is a user-facing feature with direct clinical value.

10.2 Limitations

10.2.1 Dependency on External LLM API

The system relies on Google Gemini API for all generative steps. If the API is unavailable, rate-limited, or the model changes its output schema, the pipeline fails. A fully offline or locally-deployed LLM would eliminate this dependency but requires significant hardware.

10.2.2 No EHR Integration

The current version accepts raw text input manually pasted by the user. It does not connect to any Electronic Health Record (EHR) system such as Epic or ABDM (Ayushman Bharat Digital Mission). In clinical deployment, note ingestion would need to happen via HL7 FHIR APIs or HL7 v2 interfaces.

10.2.3 Confidence Scoring is Rule-Based Only

The confidence scorer cross-checks high-salience terms (dosages, vitals, lab values) extracted by regular expressions against the original note. It does not perform deep semantic verification. Subtle paraphrasing, synonyms, or unit conversions (e.g., mmol/L to mg/dL) may not be caught, leading to false negatives in hallucination detection.

10.2.4 ROUGE Evaluation Limitations

ROUGE metrics measure lexical overlap between generated and reference summaries. They do not capture clinical accuracy, completeness, or factual correctness. A summary that is clinically correct but uses different terminology will score lower than one that is lexically similar but clinically inaccurate. BERTScore (optional dependency) provides better semantic evaluation but requires a large model download.

10.2.5 Readability Model Approximation

The Flesch-Kincaid implementation uses a vowel-group heuristic for syllable counting, which is an approximation. For medical terminology with Latin or Greek roots (e.g., 'cardiomegaly', 'hypertension'), syllable counts may be slightly inaccurate, leading to a grade level estimate that diverges from the true value. The practical impact is small but measurable.

10.2.6 No Clinical Validation

The system has not been validated by licensed medical professionals or subjected to clinical trials. Outputs should not be used for actual patient care without physician review. The confidence score is an algorithmic proxy for faithfulness to the source note — it is not a substitute for clinician oversight.

10.2.7 Section Detection May Miss Unlabelled Notes

The preprocessor uses regex-based section detection. If a clinical note does not include labelled section headers (e.g., 'Chief Complaint:', 'Assessment:'), the entire note falls back to the HPI section. This limits performance on free-form or telegraphic notes that do not follow standard templates.

11. FUTURE ENHANCEMENTS

The current version of DischargeAssist AI demonstrates a complete and functional prototype. The following enhancements would increase its clinical utility, safety, and deployment readiness in real-world healthcare settings.

11.1 EHR Integration via HL7 FHIR

The most impactful near-term enhancement would be integration with Electronic Health Record systems through the HL7 FHIR R4 standard. This would allow DischargeAssist to automatically ingest clinical notes from hospital information systems (HIS) and push generated discharge summaries back as FHIR DocumentReference or Composition resources. Integration with ABDM (Ayushman Bharat Digital Mission) would specifically align the system with India's national digital health infrastructure.

11.2 Local LLM Deployment for Data Privacy

Clinical notes contain highly sensitive Protected Health Information (PHI). A future version should support locally-deployed open-source LLMs such as Llama 3, Mistral Medical, or BioMistral as drop-in replacements for the Gemini API. This would eliminate data transmission to third-party servers and comply with hospital data governance policies and India's DPDP Act 2023.

11.3 Named Entity Recognition with Clinical NLP

The current preprocessor uses regex and a hand-crafted dictionary for drug detection and abbreviation expansion. Replacing or augmenting this with a purpose-built clinical NLP model — such as scispaCy with the 'en_core_sci_md' pipeline or Bio-BERT — would improve entity detection accuracy, especially for rare drug names, dosage formats, and complex clinical abbreviations.

11.4 Completeness Checker

A dedicated 'completeness checker' module would verify that every mandatory section of a discharge summary (chief complaint, final diagnosis, discharge medications, follow-up date) is present and non-empty. This is a lightweight but clinically meaningful validation step that would catch structurally incomplete outputs before they reach the clinician for review.

11.5 Physician Review and Feedback Loop

A structured review interface would allow physicians to accept, reject, or edit each section of the generated summary and submit feedback. Over time, these corrections could be used as fine-tuning

data to improve the LLM's domain-specific accuracy for the institution's note style — a form of institution-specific continual learning.

11.6 Structured ICD-10 / SNOMED Coding

The assessment section of the generated SOAP note currently contains free-text diagnoses. A future enhancement would map these diagnoses to standardised clinical codes (ICD-10-CM for billing, SNOMED-CT for semantic interoperability) automatically. This would enable direct use in medical billing workflows and clinical data warehouses.

11.7 Voice-to-Text Input

Many clinicians, especially in primary care, dictate their notes rather than typing them. Integrating a speech-to-text module (using Whisper or similar medical ASR models) as a pre-processing step would allow DischargeAssist to accept dictated audio as input, convert it to text, and then run the full summarisation pipeline — significantly reducing documentation burden.

11.8 PDF and DOCX Export

The current system offers copy-to-clipboard and a basic print view for the patient summary. A future version would generate a properly formatted PDF or DOCX discharge document with the hospital letterhead, patient demographics, physician signature field, and all five sections laid out in a clinical template — ready to hand to the patient or file in the record.

11.9 BERTScore and MedNLI Evaluation

Adding BERTScore as a standard (not optional) evaluation metric would provide semantic similarity measurement beyond ROUGE's lexical overlap. Further, evaluating factual consistency using the MedNLI dataset — which contains clinician-annotated natural language inference pairs from clinical notes — would give a more clinically grounded measure of output quality.

11.10 Role-Based Access Control

In a multi-user deployment, different roles (attending physician, resident, nurse, patient) would need different levels of access to the generated summary. Role-based access control (RBAC) with JWT authentication integrated into the FastAPI backend would allow the system to serve appropriate views per user role.

CONCLUSION

DischargeAssist AI demonstrates that a well-engineered, healthcare-specific Generative AI pipeline can go significantly beyond simple LLM summarisation to address real clinical documentation challenges. The project converts raw, unstructured clinical notes into complete, structured discharge summaries through a modular seven-step pipeline — preprocessing, SOAP formatting, medication extraction, follow-up drafting, patient simplification, readability adaptation, and confidence scoring — all orchestrated by a FastAPI backend and rendered in a responsive web frontend.

The novelty of the system lies not in using an LLM, but in the healthcare-specific workflow that surrounds it. The Flesch-Kincaid adaptive readability module directly addresses the well-documented health literacy gap in discharge communication. The source-grounded confidence scorer provides a safety layer that is critically absent in most LLM-based clinical tools. The multilingual translation module makes the system meaningfully applicable to the linguistically diverse Indian patient population. Together, these features produce a system that is safety-aware, patient-centred, and regionally relevant.

The project is built on open-access technologies — the Google Gemini API (free tier), FastAPI, Pydantic, SQLite, and a dependency-free ROUGE evaluator — making it reproducible and accessible without institutional resource requirements. The MTS-Dialog dataset, chosen for its open-access availability and direct alignment with the clinical dialogue-to-summary task, provides a principled evaluation benchmark.

While the system is a prototype and requires clinical validation before deployment, it establishes a technically sound and ethically responsible foundation. The audit logging, hallucination detection, and readability enforcement components reflect genuine best practices for AI in healthcare. Future work integrating EHR systems, local LLMs, clinical NLP, and physician feedback loops would progressively close the gap between this prototype and a deployment-ready clinical decision support tool.

In summary, DischargeAssist AI represents a complete, novel, and clinically motivated Generative AI application that converts documentation burden into structured, safe, and accessible patient information — a meaningful contribution to the application of AI in healthcare delivery.

APPENDIX

A. Project File Structure

GEN-AI2 PROJECT/

main.py — FastAPI application and pipeline orchestrator

models.py — Pydantic data models (SOAPNote, MedicationEntry, etc.)

preprocessor.py — Clinical note preprocessing (noise, sections, drugs, units)

prompt_templates.py — Healthcare prompt templates for each module

llm_engine.py — Google Gemini API caller (multi-model fallback, JSON repair)

soap_formatter.py — SOAP note generation module

medication_explainer.py — Medication extraction and classification module

followup_drafter.py — Follow-up instruction generation module

patient_simplifier.py — Patient-friendly summary generation module

readability.py — Adaptive Flesch-Kincaid readability scoring [NOVELTY]

confidence_scorer.py — Hallucination detection and confidence scoring [NOVELTY]

multilingual.py — Regional language translation module [NOVELTY]

evaluation.py — ROUGE-1, ROUGE-2, ROUGE-L evaluation (no ext. libraries)

database.py — SQLite audit log and discharge summary store

demo.py — Standalone pipeline demo (no server required)

dischargeassist.html — Single-file frontend (HTML/CSS/JS)

requirements.txt — Python dependencies

B. API Endpoint Reference

POST /generate — Full pipeline. Accepts raw_note, patient_id, language, target_grade. Returns complete DischargeSummary.

GET /summary/{id} — Retrieve the most recent discharge summary for a patient ID from the SQLite database.

GET /audit/{id} — Retrieve all LLM audit log entries (module, prompt hash, response, confidence) for a patient.

GET /patients — List all patient IDs that have discharge summaries stored.

GET /history — Return the most recent 8 saved discharge summaries for the history drawer.

POST /evaluate — Compute ROUGE-1, ROUGE-2, ROUGE-L, and BERTScore for a generated vs. reference summary.

GET /health — Service health check. Returns status, service name, and version.

GitHub Repository Link

(<https://github.com/gkarthik04/DischargeAssist-AI>)